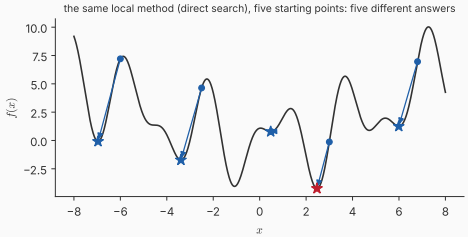
Optimization

5.1 GLODS — Global and Local Optimization using Direct Search

José Firmino Aguilar Madeira

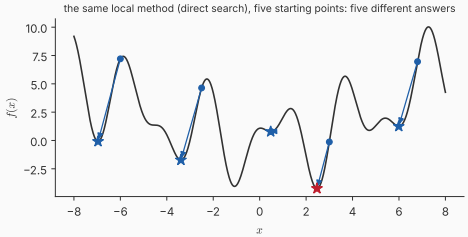
Chapter 5. Global optimization — local or global?



$f(x) = 0.1x^2 + 3\sin 2x + 2\cos(3.3x + 1)$ on $X = [-8, 8]$: the reference function for the whole chapter.

The same local method, five starting points: all five converged.

Chapter 5. Global optimization — local or global?

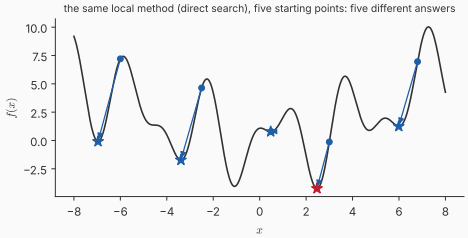


$f(x) = 0.1x^2 + 3\sin 2x + 2\cos(3.3x + 1)$ on $X = [-8, 8]$: the reference function for the whole chapter.

The same local method, five starting points: all five converged.

Question: which of them solved the problem?

Chapter 5. Global optimization — local or global?



$f(x) = 0.1x^2 + 3\sin 2x + 2\cos(3.3x + 1)$ on $X = [-8, 8]$: the reference function for the whole chapter.

The same local method, five starting points: all five converged.

Question: which of them solved the problem?

local minimum: $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for $\mathbf{x} \in X$, $\|\mathbf{x} - \mathbf{x}^*\| < \delta$ (some $\delta > 0$)

global minimum: $\boxed{f(\mathbf{x}^*) \leq f(\mathbf{x}) \text{ for all } \mathbf{x} \in X}$

Every global minimum is local; not conversely.

Converging does not mean finding the best solution of the problem.

Why search globally?

So far (Chs. 2–4) we have learned to descend and converge. In an engineering problem that is not enough:

1. **Dependence on the starting point:** a single local search may return different solutions depending on $\mathbf{x}_0$.

Why search globally?

So far (Chs. 2–4) we have learned to descend and converge. In an engineering problem that is not enough:

1. **Dependence on the starting point:** a single local search may return different solutions depending on $\mathbf{x}_0$.
2. **We do not know how many basins there are:** with a black box we do not even see the landscape.

Why search globally?

So far (Chs. 2–4) we have learned to descend and converge. In an engineering problem that is not enough:

1. **Dependence on the starting point:** a single local search may return different solutions depending on $\mathbf{x}_0$.
2. **We do not know how many basins there are:** with a black box we do not even see the landscape.
3. **One local solution may be much worse than another:** being stationary is not necessarily what the engineer wants.

Why search globally?

So far (Chs. 2–4) we have learned to descend and converge. In an engineering problem that is not enough:

1. **Dependence on the starting point:** a single local search may return different solutions depending on $\mathbf{x}_0$.
2. **We do not know how many basins there are:** with a black box we do not even see the landscape.
3. **One local solution may be much worse than another:** being stationary is not necessarily what the engineer wants.
4. **Several good minima are also information:** manufacturing, cost, robustness or maintenance may lead us to choose a solution that is not the one with the lowest f — we return to this at the end of 5.1.

But there is a price

If we do not know where the best basin is, we have to explore. And exploring costs evaluations of f.

Exploration

visiting new regions of X, looking for other basins

Refinement

descending within a basin already found, down to its minimum (exploitation)

Question: how can we explore different basins without spending the whole budget?

But there is a price

If we do not know where the best basin is, we have to explore. And exploring costs evaluations of f.

Exploration

visiting new regions of X, looking for other basins

Refinement

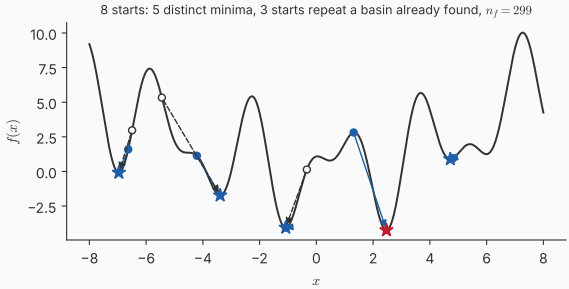
descending within a basin already found, down to its minimum (exploitation)

Question: how can we explore different basins without spending the whole budget?

The goal of this chapter

Search globally: explore different basins and try to identify solutions better than that of a single local search. Guaranteeing the global optimum is another matter.

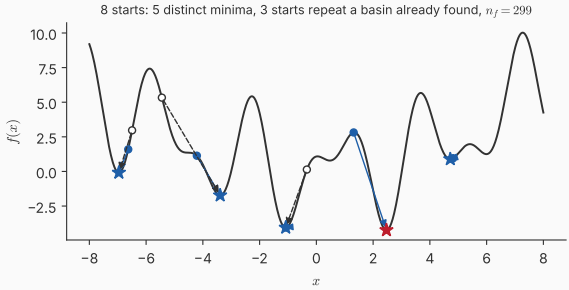
The simplest remedy: multistart



Multistart: run the same local method from several points and keep the best.

Here, 8 random starts found 5 distinct minima, including the global one.

The simplest remedy: multistart



Multistart: run the same local method from several points and keep the best.

Here, 8 random starts found 5 distinct minima, including the global one.

Question: what is the problem?

Three of the eight starts (dashed) fell into a basin already found and repeated practically the same work.

We need to explore the space without wasting too much budget.

Three ways to escape — the map of Chapter 5

How do we escape from a local basin — and how much does it cost us?

5.1 GLODS

Several points + local searches (direct search, 3.2.5) + avoiding redundancy. Deterministic (with a deterministic *search*); returns a *list* of minima.

5.2 Simulated annealing

One point that sometimes accepts getting worse.

Stochastic.

5.3 Genetic algorithms

A population: selection + recombination + mutation.

Stochastic.

At the end of each method, the same five questions:

Global exploration	**Information**	**Cost**	**Guarantee**	**Randomness**
how does it escape basins?	what does it need from f?	how many evaluations?	what can be claimed?	is a run reproducible?

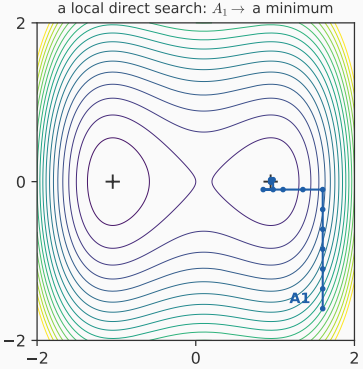
The common thread of the course — information, cost, guarantee — applied to the global question.

GLODS: multistart conditioned by proximity

GLODS

A multistart strategy conditioned by proximity: if two searches get close enough, only one remains active.

- A direct search (3.2.5) from A_1 tends to converge to the minimum of its basin.



Aluffi-Pentini on $[-2, 2]^2$ (the sequence from the 2015 presentation, redone).

Only values of f; f expensive; X compact (extreme barrier).

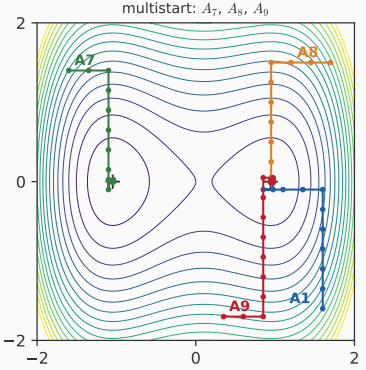
Custódio and Madeira, *J. Global Optim.* 62(1):1–28, 2015.

GLODS: multistart conditioned by proximity

GLODS

A multistart strategy conditioned by proximity: if two searches get close enough, only one remains active.

- A direct search (3.2.5) from A_1 tends to converge to the minimum of its basin.
- Multistart: searches from A_7, A_8, A_9, each run to the end.



Aluffi-Pentini on $[-2, 2]^2$ (the sequence from the 2015 presentation, redone).

Only values of f; f expensive; X compact (extreme barrier).

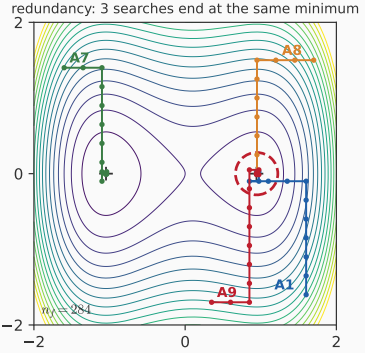
Custódio and Madeira, *J. Global Optim.* 62(1):1–28, 2015.

GLODS: multistart conditioned by proximity

GLODS

A multistart strategy conditioned by proximity: if two searches get close enough, only one remains active.

- A direct search (3.2.5) from A_1 tends to converge to the minimum of its basin.
- Multistart: searches from A_7, A_8, A_9, each run to the end.
- Redundancy: three end at the same minimum; $n_f = 284$ for 2 minima.



Aluffi-Pentini on $[-2, 2]^2$ (the sequence from the 2015 presentation, redone).

Only values of f; f expensive; X compact (extreme barrier).

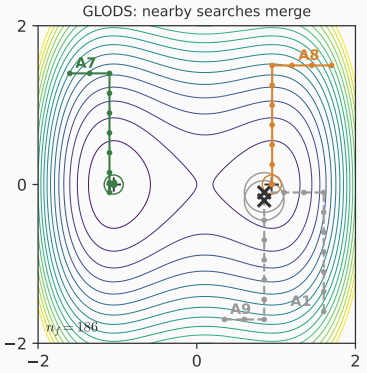
Custódio and Madeira, *J. Global Optim.* 62(1):1–28, 2015.

GLODS: multistart conditioned by proximity

GLODS

A multistart strategy conditioned by proximity: if two searches get close enough, only one remains active.

- A direct search (3.2.5) from A_1 tends to converge to the minimum of its basin.
- Multistart: searches from A_7, A_8, A_9, each run to the end.
- Redundancy: three end at the same minimum; $n_f = 284$ for 2 minima.
- GLODS: the same searches simultaneously; the one that enters the neighbourhood of a better one is no longer active ($A_9 \to A_1$, $A_1 \to A_8$). The same 2 minima, $n_f = 186$.



Aluffi-Pentini on $[-2, 2]^2$ (the sequence from the 2015 presentation, redone).

Only values of f; f expensive; X compact (extreme barrier).

Custódio and Madeira, *J. Global Optim.* 62(1):1–28, 2015.

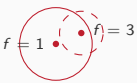
The idea that sets GLODS apart: $(\mathbf{x}, \alpha, r, i)$

Each point in the list stores four things:

$\boxed{\mathbf{x}}$ position $\quad$ $\boxed{\alpha}$ direct-search step size $\quad$ $\boxed{r}$ comparison radius $\quad$ $\boxed{i}$ active / inactive



far apart: two active searches



close: one falls within the other's radius



only the better one stays active

Rule (simple decrease)

A new point $\mathbf{x}_{\text{new}}$ is comparable with a stored point $\mathbf{y}$ when $\|\mathbf{x}_{\text{new}} - \mathbf{y}\| \leq r_{\mathbf{y}}$. Comparable with none: new region, enters active. Better than all comparable points: enters active and deactivates the worse ones. Some comparable point as good or better: enters inactive (if it is better than some active one) or does not enter at all. An active point may become inactive; never the reverse. Only active points are *poll* centres.

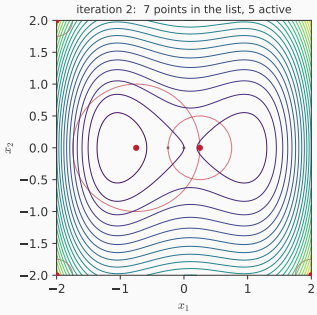
The radius comes from the direct-search step size, not from a probabilistic construction: $r =$ step with which the point entered ($\alpha \|\mathbf{d}\|$; α with unit directions); it is updated by setting $r = \max(r, \alpha)$, so it does not decrease when α decreases and $r \geq \alpha$.

When two searches get close: merging

The full GLODS, with a 2^n-centres *search*, on the same problem: the new points go through the comparison rule and the searches that get close to better ones are no longer active.

Three types of iteration

Successful: a new active point entered — the step is kept or increased (in the examples, $\times 2$). Unsuccessful: no point entered — the centre's step is reduced (in the examples, $\times \frac{1}{2}$). Merging: only inactive points entered — not a failure: proximity detected redundant work.



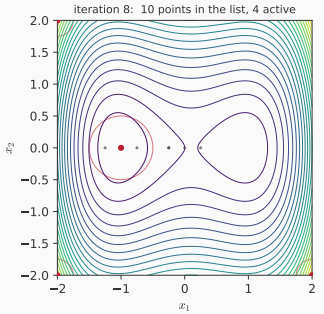
Aluffi-Pentini, $\alpha_0 = r_0 = 0.25$. Red: active, with their radius; grey: inactive.

When two searches get close: merging

The full GLODS, with a 2^n-centres *search*, on the same problem: the new points go through the comparison rule and the searches that get close to better ones are no longer active.

Three types of iteration

Successful: a new active point entered — the step is kept or increased (in the examples, $\times 2$). Unsuccessful: no point entered — the centre's step is reduced (in the examples, $\times \frac{1}{2}$). Merging: only inactive points entered — not a failure: proximity detected redundant work.



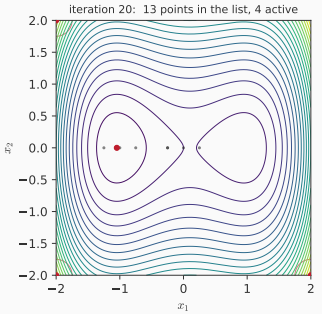
Aluffi-Pentini, $\alpha_0 = r_0 = 0.25$. Red: active, with their radius; grey: inactive.

When two searches get close: merging

The full GLODS, with a 2^n-centres *search*, on the same problem: the new points go through the comparison rule and the searches that get close to better ones are no longer active.

Three types of iteration

Successful: a new active point entered — the step is kept or increased (in the examples, $\times 2$). Unsuccessful: no point entered — the centre's step is reduced (in the examples, $\times \frac{1}{2}$). Merging: only inactive points entered — not a failure: proximity detected redundant work.



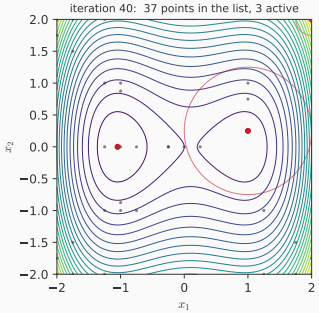
Aluffi-Pentini, $\alpha_0 = r_0 = 0.25$. Red: active, with their radius; grey: inactive.

When two searches get close: merging

The full GLODS, with a 2^n-centres *search*, on the same problem: the new points go through the comparison rule and the searches that get close to better ones are no longer active.

Three types of iteration

Successful: a new active point entered — the step is kept or increased (in the examples, $\times 2$). Unsuccessful: no point entered — the centre's step is reduced (in the examples, $\times \frac{1}{2}$). Merging: only inactive points entered — not a failure: proximity detected redundant work.



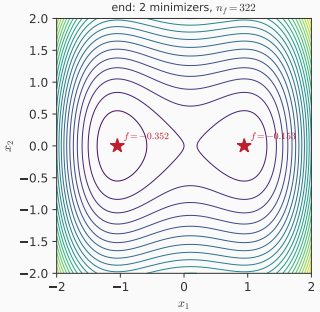
Aluffi-Pentini, $\alpha_0 = r_0 = 0.25$. Red: active, with their radius; grey: inactive.

When two searches get close: merging

The full GLODS, with a 2^n-centres *search*, on the same problem: the new points go through the comparison rule and the searches that get close to better ones are no longer active.

Three types of iteration

Successful: a new active point entered — the step is kept or increased (in the examples, $\times 2$). Unsuccessful: no point entered — the centre's step is reduced (in the examples, $\times \frac{1}{2}$). Merging: only inactive points entered — not a failure: proximity detected redundant work.

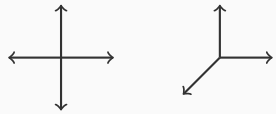


Aluffi-Pentini, $\alpha_0 = r_0 = 0.25$. Red: active, with their radius; grey: inactive. End: 2 minimizers, $n_f = 322$; 32 successful it., 29 unsuccessful, 10 merging. Multistart from 8 random points: in one run, the same 2 ($n_f = 848$); in 30 runs, $n_f = 850$ [823, 882] and both only in 12.

Poll: the local component (direct search, 3.2.5)

From $\mathbf{x}_k$ with step α_k, evaluate f at the poll points $\mathbf{x}_k + \alpha_k \mathbf{d}$, $\mathbf{d} \in D_k$, where D_k is a positive spanning set: every vector of $\mathbb{R}^n$ is a *nonnegative* linear combination of elements of D_k. If $\nabla f(\mathbf{x}_k) \neq \mathbf{0}$, some direction of D_k is a descent direction — without computing the gradient.

- Maximal positive basis $D_\oplus = [I \quad -I]$: $2n$ directions $(\pm \mathbf{e}_i)$; minimal: $n+1$ directions.
- Success (some poll point improves): the step is kept or grows, $\alpha_{k+1} \in [\alpha_k, \gamma \alpha_k]$; failure: it decreases, $\alpha_{k+1} \in [\beta_1 \alpha_k, \beta_2 \alpha_k]$, $0 < \beta_1 \leq \beta_2 < 1$. In the examples: $\times 2$ and $\times \frac{1}{2}$.
- Bounds $\mathbf{x}^{(L)} \leq \mathbf{x} \leq \mathbf{x}^{(U)}$ by *extreme barrier*: $f = +\infty$ outside X (f is not evaluated; see 4.3.3; not to be confused with the logarithmic barrier of 4.3.2).
- Local stopping: $\alpha_k \leq \alpha_{\min}$. The poll step plays the role of the Δ_i of Hooke–Jeeves (it is not the λ_k of Ch. 3); it returns in 6.5 (DMS).



$D_\oplus = [I \quad -I]$: $2n$ minimal: $n+1$

Cost per iteration

$n_f \leq |D_k|$ ($2n$ with $D_\oplus$); only values of f. It is the "thermostat" of Hooke–Jeeves and Box; in the formalized versions (GPS, 3.2.5; here in GLODS) — not in the simplified Box of 3.2.4 — it gives stationarity results under suitable assumptions, which NM does not have.

Search: the global component

As in modern direct-search methods (`patternsearch`), the iteration is organized in two steps:

Search (optional, global)

Evaluate f at *any* finite set of points of the domain, looking for promising regions not yet explored: random sampling, Latin hypercube, Sobol/Halton sequences, or 2^n-centres (next page). If it generates a new active point, the iteration is successful and ends without a *poll*. It need not be done at every iteration: in the paper, the *search* is performed when the number of active points not yet identified as local minimizers is equal to 1.

Poll (local, structured)

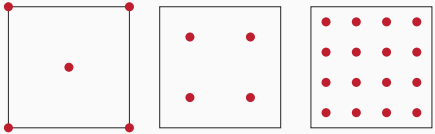
Explores an active region: the poll $\mathbf{x}_k + \alpha_k \mathbf{d}$, $\mathbf{d} \in D_k$, around the active point chosen as centre (the one with the lowest f still to be explored). It is this step, with the structure of positive spanning sets and of the step size α_k, that supports the convergence theory; the *search* can be anything.

Every new point, whether from the *search* or the *poll*, goes through the same comparison rule ("The idea that sets GLODS apart"): it decides whether the point enters active, inactive, or not at all.

One option for the *search*: 2^n-centres

A deterministic strategy that meets three requirements: points asymptotically dense in X, uniform coverage and reproducibility.

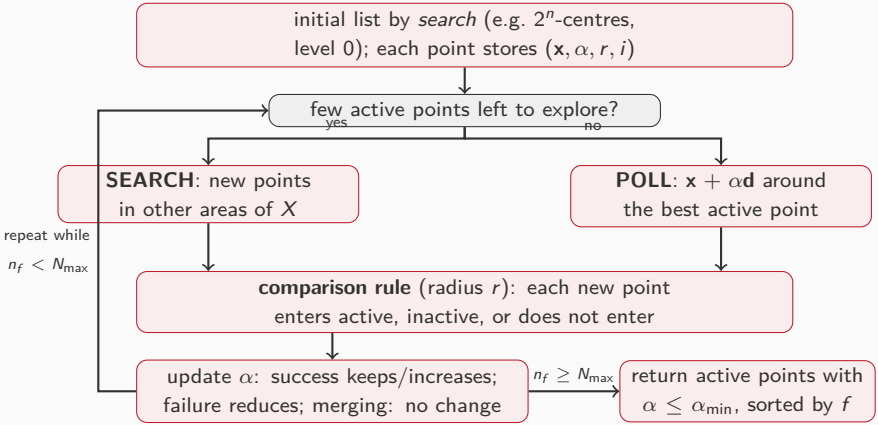
- **Level 0**: the centre *and the vertices* of the box ($2^n + 1$ points).
- **Level** $\ell \geq 1$: the box is subdivided into $(2^\ell)^n$ boxes; the points are their centres.
- To keep the cost from exploding, each *search* uses only a subset of 2^n centres not yet used; the rest are left for the following ones.
- Each new centre goes through the comparison rule (those that fall within the radius of a better point enter inactive or not at all). In the teaching version they enter with α_0, r_0; in the full algorithm, comparable points may inherit the parameters of the existing search.



$n = 2$: level 0, 5 points (centre + vertices); level 1, 4 centres; level 2, 16 centres, served 4 at a time.

Other *search* strategies: random sampling, Latin hypercube (LHS), Sobol and Halton sequences. GLODS is not the 2^n-centres: it is the *search*/*poll* framework with proximity; the *search* is a choice.

The full GLODS



The *poll* ensures convergence (theory); the *search* looks for new basins; the comparison rule avoids repeated work.

Algorithm (teaching version)

GLODS: X, α_0, $r_0 \geq d_{\max}\alpha_0$, $\alpha_{\min}$, $N_{\max}$

1: set L to the initial *search* list (level-0 2^n-centres), each point with α_0, r_0, after the comparison rule
2: **while** $n_f < N_{\max}$ **do**
3: set $C = \{\mathbf{x} \in L \text{ active} : \alpha_{\mathbf{x}} > \alpha_{\min}\}$ ▷ active points not yet identified as minima
4: **if** $|C| \leq 1$ **then** *search*: 2^n new centres into L; if one entered active, **continue**; otherwise recompute C
5: **end if**
6: **if** $C = \emptyset$ **then continue** (next *search*) or stop, if levels/budget are exhausted
7: **end if**
8: set $\mathbf{x} = \arg\min_C f$ ▷ *poll* centre: the best one still to explore
9: *poll*: for each $\mathbf{d} \in D$, submit $\mathbf{x} + \alpha_{\mathbf{x}}\mathbf{d}$ to the comparison rule (step $\alpha_{\mathbf{x}}$, radius $\alpha_{\mathbf{x}} \|\mathbf{d}\|$)
10: **if** some active point entered **then** successful: set $\alpha = \gamma\alpha$, $r = \max(r, \alpha)$ at the new active points
11: **else if** some inactive point entered **then** merging: nothing changes
12: **else** unsuccessful: set $\alpha_{\mathbf{x}} = \beta\alpha_{\mathbf{x}}$
13: **end if**
14: **end while**
15: **return** active points with $\alpha \leq \alpha_{\min}$, sorted by f ▷ local minimizers

- The output is a list, not a point: the first is the best minimum found. The cost is n_f; the budget $N_{\max}$ is what the user controls.
- r = step with which the point entered ($\alpha \|\mathbf{d}\|$; α with unit directions); it is updated by setting $r = \max(r, \alpha)$, hence $r \geq \alpha$. In the paper, $r_0 \geq d_{\max}\alpha_0$.
- Deterministic with 2^n-centres; stochastic with random sampling (then: several runs, median).
- Teaching version: simple decrease, no greedy ordering of the list and no initial line sampling; the structure is that of the paper.

What can be guaranteed?

Theorem (refining subsequences) — Custódio and Madeira (2015)

Under the assumptions of the globalization strategy used (integer lattices with simple decrease, or sufficient decrease with a forcing function), X compact and f bounded below: $\liminf \alpha_k = 0$, and there is a convergent subsequence of unsuccessful *poll* iterates with $\alpha_k \to 0$. (Globalization in the sense of 3.3.1: from any $\mathbf{x}_0$, towards stationarity, not towards the global minimum.)

Theorem (stationarity)

If f is L-Lipschitz (R) near the limit $\mathbf{x}_*$ and the refining directions are dense in the tangent cone, $\mathbf{x}_*$ is Clarke stationary: $f^\circ(\mathbf{x}_*; \mathbf{d}) \geq 0$ for the directions of the tangent cone to X. If f is strictly differentiable at $\mathbf{x}_*$: in the interior $\nabla f(\mathbf{x}_*) = \mathbf{0}$; on the bounds, KKT. With a fixed $D_\oplus$ (as in the examples) only $f^\circ(\mathbf{x}_*; \mathbf{d}) \geq 0$ for the $\mathbf{d} \in D_\oplus$ that do not leave X (in the interior, all) is guaranteed; with strict differentiability this is enough. For f only Lipschitz, dense directions are needed.

$f^\circ(\mathbf{x}; \mathbf{d}) = \limsup_{\mathbf{y} \to \mathbf{x},\ t \downarrow 0} [f(\mathbf{y} + t\mathbf{d}) - f(\mathbf{y})]/t$: Clarke generalized directional derivative. Tangent cone to X at $\mathbf{x}$: the directions along which one enters X from $\mathbf{x}$. In the line of Torczon, Audet and Dennis, Kolda–Lewis–Torczon; full presentation.

What this says — and what it does not

- The theory provides stationarity conditions for limit points of refining subsequences; applying the method does not require smoothness (only the theorem requires local Lipschitz continuity).
- Global exploration comes from the *search*: how many basins are found depends on the budget, the sampling and the comparison radius. Without additional assumptions on f, there is no general guarantee of identifying exactly the global minimum with a finite budget.
- Noise: GLODS does not need derivatives, which noise would make unreliable; but this is *not* a theory for noisy functions.

What the 2015 paper showed

61 bound-constrained problems, $2 \leq n \leq 10$, budget 20 000, stopping at $\alpha < 10^{-8}$; data profiles against **DIRECT**, **MCS** and a pure multistart. The conclusion is balanced:

- pure multistart: poor performance (the waste seen at the start of the deck); DIRECT behind MCS and GLODS;
- *small* budgets: MCS was the best on the test set;
- GLODS competitive, with the best performance for *moderate* budgets; with random initialization, the advantage of including a *search* became clear — 2^n-centres, Halton and Sobol performed well;
- and the additional feature: looking for the different local minima, not just the best value.

Problem (Tab. 6)	n	known min.	local	found
Becker–Lago	2	4		4
Branin–Hoo	2	3		2–3
Aluffi-Pentini	2	2		1
Shekel-45	4	5		4–5
Six-hump camel	2	6		2
Periodic	2	50		6–8

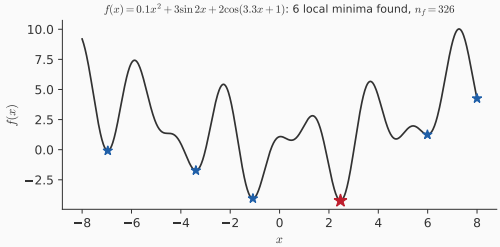
Variants 2^n-centres, Sobol, Halton, with $\alpha_0 = r_0 = 1$ on the original domains. GLODS tries to explore different basins; budget, *search* and comparison radius determine how many it can identify. On Aluffi-Pentini the paper found 1 of the 2; in the teaching example, varying r_0 ($r_0 = 1$ gives 1, $r_0 = 0.25$ gives 2) shows how the radius can influence the separation of basins.

GLODS on the Chapter 5 function

$f(x) = 0.1x^2 + 3\sin 2x + 2\cos(3.3x + 1)$ on $[-8, 8]$; 2^n-centres up to level 4, $\alpha_0 = r_0 = 1$, $\alpha_{\min} = 10^{-4}$, $n_f = 326$:

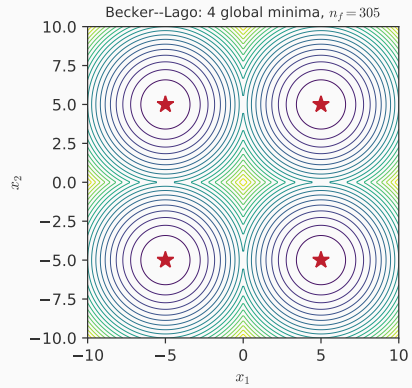
	x	$f(x)$
1	2.4680	−4.2381
2	−1.0819	−4.0531
3	−3.3918	−1.7273
4	−6.9615	−0.0849
5	5.9908	1.2485
6	8.0000	4.2533

- The minima at 0.47 and 4.72 were absorbed: points from those basins fell within the comparison radius of neighbours with lower f. Depth alone does not decide (4.72 is lower than 5.99): order and radius also count.



- Five of the seven interior local minima, with the global one $x^* = 2.468$, $f^* = -4.238$ first.
- $x = 8$: boundary minimum (f decreases up to the bound; stationary in the Clarke–KKT sense).

Several global minima: Becker–Lago



$f = (|x_1| - 5)^2 + (|x_2| - 5)^2$ on $[-10, 10]^2$.

- Four global minima, all with $f = 0$, at $(\pm 5, \pm 5)$.
- GLODS finds all four with $n_f = 305$ ($\alpha_0 = r_0 = 2$) — as in the paper (4 of 4).
- f is nondifferentiable on the axes: the algorithm applies without derivatives; Clarke analysis is the appropriate framework for interpreting stationarity.
- A method that returns a single point would give *one* of the four — and we would not know there are three others.

On the computer: the GLODS MATLAB code

The original code (version 0.2) is distributed in class. It is not meant to be read from end to end: what matters is recognizing the pieces of the method in it.

Idea of the method	File
search (global component)	`search_step.m`
proximity: comparison radius and list of active/inactive points	`match_point.m`, `merge.m`
main algorithm (*search*/*poll*, update of α)	`glods.m`
options (*search* strategy, α_0, r_0, stopping, budget)	`parameters_glods.m`
starting point: a simple call on a test problem	`driver_glods.m`

- Start with `driver_glods.m`: run it, change the *search* strategy and the initial radius, and see how many minima appear in the list.
- Only then open `glods.m`, with the flowchart "The full GLODS" alongside.

OPTIMIZATION IN PRACTICE — fibre orientation in a composite

Optimization of the fibre orientation in a laminated composite specimen (Monte, Infante, Madeira, Moleiro, 2017)

Decision the fibre orientation in each of the 8 plies: angles in $[-85°, 90°]$, in increments of $5°$ (36 values per ply).

Objective minimize the maximum displacement of the specimen under load.

Information on f values only: each evaluation is a finite element analysis (ANSYS), called by GLODS — black box, expensive, no derivatives.

Result a list of 14 solutions with maximum displacements between 3.17 and 3.60 mm — all of them solve the problem.

The final decision was not "the one with the lowest f": among solutions with equivalent performance, the easiest to manufacture (lay up) was chosen. A method that returned only one solution would not naturally provide this set of alternatives in a single run. Reference: *Mechanics of Advanced Materials and Structures* 24(5):410–416, 2017, DOI 10.1080/15376494.2016.1191099.

Is the global minimum always the chosen solution?

Two of the 14 composite solutions:

Solution 12 — the best f

$[-35/45/40/80/-40/50/-55/40]$

maximum displacement 3.17 mm

eight different orientations

Solution 1

$[45/-50/-50/45/45/-50/-50/45]$

maximum displacement 3.40 mm $(+7\%)$

only two orientations, in a symmetric pattern

Is the global minimum always the chosen solution?

Two of the 14 composite solutions:

Solution 12 — the best f

$[-35/45/40/80/-40/50/-55/40]$
maximum displacement 3.17 mm
eight different orientations

Solution 1

$[45/-50/-50/45/45/-50/-50/45]$
maximum displacement 3.40 mm (+7%)
only two orientations, in a symmetric pattern

Question: which one would you rather lay up?

Is the global minimum always the chosen solution?

Two of the 14 composite solutions:

Solution 12 — the best f

$[-35/45/40/80/-40/50/-55/40]$
maximum displacement 3.17 mm
eight different orientations

Solution 1

$[45/-50/-50/45/45/-50/-50/45]$
maximum displacement 3.40 mm (+7%)
only two orientations, in a symmetric pattern

Question: which one would you rather lay up?

The mathematical optimum is the best according to the model we formulated.
The engineering decision may use information that did not enter the model.

If we choose another solution for cost, manufacturing or materials, this reveals that there were relevant criteria outside f. If the algorithm had returned only solution 12, we would not even know that solution 1 existed.

So why find several minima?

Because different basins may correspond to qualitatively different design alternatives.

- **Similar performance** $\rightarrow$ nearly equivalent alternatives.
- **Manufacturing, cost, materials** $\rightarrow$ criteria that are not in f.
- **Robustness of the solution** (not of the method) $\rightarrow$ a wide basin (small changes in $\mathbf{x}$ barely degrade f) may be preferable to a slightly better narrow minimum.
- **Later decision** $\rightarrow$ the engineer chooses among alternatives, list in hand.

An algorithm that returns only the best point answers "what is the best solution according to f?". A list also answers "what alternatives do I have?".

So why find several minima?

Because different basins may correspond to qualitatively different design alternatives.

- **Similar performance** → nearly equivalent alternatives.
- **Manufacturing, cost, materials** → criteria that are not in f.
- **Robustness of the solution** (not of the method) → a wide basin (small changes in $\mathbf{x}$ barely degrade f) may be preferable to a slightly better narrow minimum.
- **Later decision** → the engineer chooses among alternatives, list in hand.

An algorithm that returns only the best point answers "what is the best solution according to f?". A list also answers "what alternatives do I have?".

And if those criteria are important enough?

Perhaps they should not stay outside the model: they become objectives.

$$\text{performance} \longrightarrow (\text{performance, cost, manufacturability, } \ldots)$$

⇒ Ch. 6: several objectives.

So why find several minima?

Because different basins may correspond to qualitatively different design alternatives.

- **Similar performance** → nearly equivalent alternatives.
- **Manufacturing, cost, materials** → criteria that are not in f.
- **Robustness of the solution** (not of the method) → a wide basin (small changes in $\mathbf{x}$ barely degrade f) may be preferable to a slightly better narrow minimum.
- **Later decision** → the engineer chooses among alternatives, list in hand.

An algorithm that returns only the best point answers "what is the best solution according to f?". A list also answers "what alternatives do I have?".

And if those criteria are important enough?

Perhaps they should not stay outside the model: they become objectives.

$$\text{performance} \longrightarrow (\text{performance, cost, manufacturability, } \dots)$$

⇒ Ch. 6: several objectives.

And with several objectives?

MultiGLODS extends this global–local idea to multiobjective optimization (Custódio and Madeira, *J. Global Optim.* 72, 2018). We do not study it in this course; we return to several objectives in Ch. 6.

5.1 GLODS — summary table

What information do we have about f? What does it cost to obtain? What can we guarantee?

Information	only values of f (order 0); bounds $\mathbf{x}^{(L)} \leq \mathbf{x} \leq \mathbf{x}^{(U)}$ by extreme barrier
Global exploration	several local direct searches (*poll*) + *search* in unexplored areas, without duplicating nearby searches: comparison radius and list of active/inactive points
Cost	evaluations of f: $n_f \leq \lvert D \rvert$ per iteration ($2n$ with $[I \ -I]$) plus those of the *search*; the budget $N_{\max}$ is the user's decision
Guarantee	(Clarke) stationarity of limit points of refining subsequences, if f is Lipschitz near the limit and the refining directions are dense (with a fixed $D_{\oplus}$: only along the directions of $D_{\oplus}$); no guarantee of finding all basins with finite n_f
Output	a *list* of local-minimizer candidates, sorted by f; allows deciding by criteria that were not in f
Randomness	can be deterministic (2^n-centres; deterministic sequences) or stochastic (random sampling, LHS); in the latter case it depends on the seed and requires several runs
Limitation	high dimension ($2n$ poll points per iteration, 2^n-centres) and very expensive functions require managing the budget; small basins may be absorbed by the radius

KEY TAKEAWAYS — 5.1

1. **Global exploration:** local *poll* + *search* in new areas; the comparison radius avoids repeating searches.
2. **Information and cost:** only values of f; cost in evaluations of f, with a fixed budget.
3. **Guarantee:** (Clarke) stationarity of refining limits, if f is Lipschitz near the limit and the refining directions are dense (with a fixed $D_{\oplus}$: only along the directions of $D_{\oplus}$); no guarantee of finding all basins.
4. **Output:** a list of local-minimizer candidates — so that we can also choose by criteria outside f.
5. **Randomness:** deterministic with a deterministic *search*; otherwise, several runs.

Next (5.2, 5.3): randomness — SA accepting uphill moves, the GA with a population.

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Custódio, A. L., Madeira, J. F. A., *GLODS: Global and Local Optimization using Direct Search*, Journal of Global Optimization 62(1):1–28, 2015 — the GLODS paper.
- Conn, A. R., Scheinberg, K., Vicente, L. N., *Introduction to Derivative-Free Optimization*, SIAM (MPS-SIAM Series on Optimization 8), 2009 — Ch. 7.
- Madeira, J. F. A., *Multiobjective and global optimization with directional direct search*, PhD thesis in Mathematics, IST, Universidade de Lisboa, 2019.
- Horst, R., Tuy, H., *Global Optimization: Deterministic Approaches*, 3rd ed., Springer, 1996.

Resources

- Extensions: Madeira (2026) (GLODS-SI, scale invariance); *GLODS-SI-Mix* — continuous, discrete and categorical variables simultaneously (submitted).
- The **full presentation** (98 pages): convergence with Clarke–Jahn derivatives, data profiles against DIRECT and MCS. These lecture notes are the teaching version.
- GLODS v0.2 (MATLAB), distributed in class with the technical report (files in "On the computer"); page: `docentes.fct.unl.pt/algb/pages/glods`.
- IST theses: Luís (2022, GLODS and SA), Figueiredo and Mareco (2016) — *Course bibliography*.